



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 10

Lectura de Valores Analógicos con un Potenciómetro en Raspberry Pi Pico W

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

3 de Marzo de 2026

Índice

1. Introducción	2
1.1. Objetivo	2
1.2. Trabajo Previo	2
1.2.1. Señales Analógicas y Digitales	2
1.2.2. Proceso de Conversión Analógica a Digital	2
1.2.3. ADC en la Raspberry Pi Pico W	3
2. Desarrollo	3
2.1. Diseño del Circuito	3
2.2. Descripción del Sistema Extendido	4
2.3. Proceso de Desarrollo con PlatformIO	5
2.4. Errores Comunes y Soluciones	6
3. Preguntas de Repaso	6
3.1. Si el voltaje del potenciómetro es 1 V, ¿qué valor de ADC representará?	6
3.2. ¿Cuál es la representación binaria de un voltaje de 2 V?	7
3.3. ¿Por qué se utiliza 4095 y no 4096 en la ecuación del ADC?	7
3.4. ¿Cuál es la resolución de un ADC de 16 bits?	7
3.5. Conversión de valores binarios del ADC a voltaje	7
3.6. Funcionamiento del Código C/C++ (Arduino)	8
4. Conclusión	9
Anexos	9

1. Introducción

En esta práctica se estudia el funcionamiento de un **convertidor analógico a digital** (ADC) integrado en la Raspberry Pi Pico W, utilizando un **potenciómetro de $5\text{ k}\Omega$** como fuente de señal analógica variable. El objetivo central es comprender cómo un microcontrolador, cuyas entradas son inherentemente digitales, puede interpretar magnitudes continuas del mundo físico mediante el proceso de conversión ADC, y cómo representar los valores obtenidos tanto en el monitor serial como en una **interfaz web en tiempo real**.

Como extensión práctica del esquema básico de lectura ADC, el proyecto implementa un servidor HTTP embebido en la Pico W que sirve una **gráfica interactiva** con Chart.js, accesible desde cualquier dispositivo conectado a la misma red WiFi. Adicionalmente, un **LED** varía su frecuencia de parpadeo en función del voltaje leído, y un **botón** permite activar un modo *HOLD* que congela la lectura, emulando el comportamiento de un multímetro digital.

1.1. Objetivo

Entender el funcionamiento de un convertidor analógico a digital, aprender a leer valores analógicos desde un potenciómetro, interpretar los valores convertidos como señal digital, y utilizar el monitor serial y un servidor web para visualizar la información del microcontrolador en tiempo real.

1.2. Trabajo Previo

1.2.1. Señales Analógicas y Digitales

Una **señal analógica** es aquella representable por una función matemática continua, como la temperatura, presión o luz. Sus características principales son que pueden procesarse directamente en tiempo real, cuentan con parámetros de amplitud y frecuencia, y son de alta densidad informativa.

Una **señal digital**, en cambio, es de tipo discreto: solo toma valores en intervalos definidos, generalmente representados como ceros y unos en sistema binario. Sus parámetros son la altura de pulso (nivel eléctrico), duración (ancho de pulso) y frecuencia de repetición.

1.2.2. Proceso de Conversión Analógica a Digital

La conversión ADC comprende cuatro etapas fundamentales:

1. **Muestreo:** Se toma la señal analógica y se mide en intervalos de tiempo regulares. La frecuencia de muestreo debe cumplir el *Teorema de Nyquist*: debe ser al menos el doble de la frecuencia más alta presente en la señal para evitar *aliasing*.
2. **Cuantización:** Cada muestra se asigna a un valor discreto dentro de un rango determinado. La precisión depende de la resolución del convertidor: un ADC de 8 bits tiene 256 niveles; uno de 12 bits tiene 4096 niveles.

3. **Codificación:** Los valores cuantizados se convierten en código binario, representando digitalmente el valor de la señal en cada instante de muestreo.
4. **Procesamiento digital (opcional):** Se aplica filtrado o compresión para mejorar la calidad de la señal resultante.

1.2.3. ADC en la Raspberry Pi Pico W

La Raspberry Pi Pico W cuenta con un **ADC de 12 bits**, accesible en los pines GP26, GP27 y GP28, con un voltaje máximo de referencia de 3,3 V. El rango de valores del ADC es de 0 a 4095. La fórmula para convertir el valor ADC a voltaje es:

$$V_{pot} = \text{Valor ADC} \times \frac{3,3}{4095} \quad (1)$$

Y despejando para obtener el valor ADC a partir del voltaje:

$$\text{Valor ADC} = V_{pot} \times \frac{4095}{3,3} \quad (2)$$

La resolución del ADC de 12 bits es:

$$\text{Resolución} = \frac{3,3 \text{ V}}{4095} \approx 0,8 \text{ mV por nivel} \quad (3)$$

2. Desarrollo

2.1. Diseño del Circuito

El circuito utiliza el **canal ADC0** de la Pico W (pin GP26) para leer la señal analógica del potenciómetro de 5 kΩ. Las conexiones son las siguientes:

- **Potenciómetro:** La pata izquierda se conecta a GND en serie con una **resistencia de protección** para limitar el voltaje mínimo y proteger el pin ADC. La pata derecha se conecta al pin de 3,3 V (3V3OUT). La pata central (wiper) se conecta a **GP26 (ADC0)**.
- **LED:** Conectado al pin **GP14** con una resistencia limitadora de corriente de 220 Ω en serie hacia GND. Su frecuencia de parpadeo varía proporcionalmente al voltaje leído.
- **Botón:** Conectado al pin **GP15** con resistencia *pull-up* interna activa (INPUT_PULLUP); la otra pata al GND. Activo en nivel bajo.
- **Alimentación:** La placa se alimenta por USB para poder visualizar la señal serial y utilizar la conectividad WiFi.

El efecto de la resistencia de protección en serie con GND crea un divisor de tensión que establece un voltaje mínimo distinto de cero. Con una resistencia de $100\ \Omega$:

$$V_{min} = 3,3 \times \frac{100}{5000 + 100} \approx 0,065\text{ V} \rightarrow \text{ADC} \approx 80 \quad (4)$$

Esto significa que el rango real del ADC no comienza en 0, sino en un valor pequeño, comportamiento visible en la gráfica web como una línea que no toca el eje inferior.

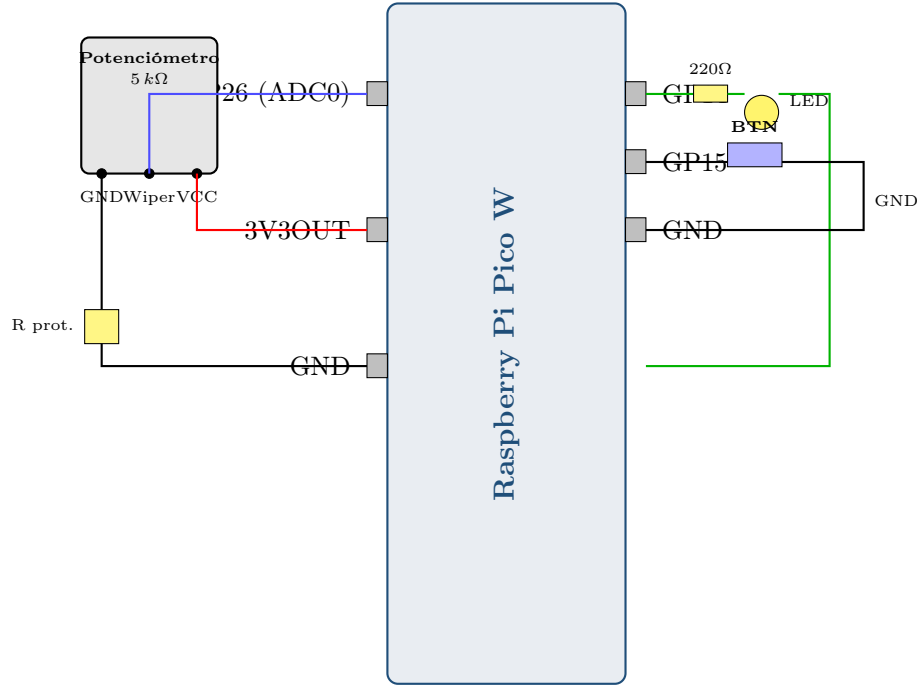


Figura 1: Diagrama esquemático del circuito: el potenciómetro conecta su wiper a GP26 (ADC0); el LED en GP14 varía su parpadeo según el voltaje; el botón en GP15 activa el modo HOLD.

2.2. Descripción del Sistema Extendido

Además de la lectura básica del ADC descrita en el documento de la práctica, el sistema implementado incorpora tres funcionalidades adicionales:

Servidor Web con gráfica en tiempo real: La Pico W actúa como punto de acceso HTTP en el puerto 80. Al conectarse a su dirección IP desde un navegador, se sirve una página HTML con Chart.js que consulta el endpoint `/data` cada 500 ms, actualizando automáticamente una gráfica de línea con las últimas 60 muestras de voltaje.

LED con frecuencia variable: El intervalo de parpadeo del LED en GP14 se mapea inversamente al valor del ADC mediante la expresión:

$$t_{blink} = t_{max} - \frac{\text{ADC}}{4095} \times (t_{max} - t_{min}) \quad (5)$$

donde $t_{min} = 50$ ms y $t_{max} = 1000$ ms. Con el potenciómetro al mínimo el LED parpadea lento; al máximo, parpadea rápido.

Botón HOLD: Al presionar el botón, el sistema entra en modo pausa: el LED queda encendido fijo, la gráfica web se congela en el último valor registrado y el monitor serial muestra los valores congelados. Un segundo clic reanuda la operación. Esta funcionalidad emula el botón *HOLD* de un multímetro digital.

2.3. Proceso de Desarrollo con PlatformIO

1. **Creación del proyecto:** En PlatformIO se creó la carpeta `Practica_10` seleccionando la placa *Raspberry Pi Pico W* con framework *Arduino* usando la plataforma `maxgerhardt/platform-raspberrypi` con el core de Earle Philhower.
2. **Configuración del ADC:** Se llamó `analogReadResolution(12)` en `setup()` para establecer resolución de 12 bits (0–4095), equivalente al ADC nativo de la Pico W.
3. **Conexión WiFi y servidor:** Se inicializó `WiFi.begin()` con las credenciales de red y se registraron dos rutas: `/` para la página HTML y `/data` para el JSON con los datos del ADC.
4. **Envío chunked del HTML:** El HTML de la interfaz web se envía en fragmentos usando `server.setContent()` con `CONTENT_LENGTH_UNKNOWN` para evitar saturar la RAM limitada de la Pico W al construir un string completo.
5. **Lógica no bloqueante:** Se utilizó `millis()` para controlar el muestreo del ADC, el parpadeo del LED y el anti-rebote del botón de forma concurrente, sin bloquear el procesamiento de peticiones HTTP en `server.handleClient()`.
6. **Compilación y carga:** Se compiló con *Build* y se cargó en modo BOOTSEL mediante *Upload*. La IP asignada por el router aparece en el Monitor Serial al conectarse exitosamente.

2.4. Errores Comunes y Soluciones

Cuadro 1: Errores encontrados y soluciones aplicadas durante el desarrollo

Error Encontrado	Solución Aplicada
Monitor serial muestra puntos continuos sin conectar al WiFi.	Las credenciales estaban siendo tomadas de las <code>build_flags</code> con escapes incorrectos. Se definieron directamente en el código fuente con <code>#define WIFI_SSID "..."</code> eliminando el problema de escape de comillas.
La dirección IP carga pero el navegador no muestra la página.	El HTML completo se intentaba enviar como un único <code>String</code> , agotando la heap de la Pico W. Se reescribió usando <code>setContentLength(CONTENT_LENGTH_UNKNOWN)</code> y múltiples llamadas a <code>sendContent()</code> en fragmentos más pequeños.
El WiFi no conecta con red disponible (puntos infinitos).	El router tiene banda 5 GHz activa. La Pico W solo soporta 2.4 GHz . Se seleccionó explícitamente la SSID de la banda 2.4 GHz del router.
El botón registra múltiples cambios de estado por un solo clic.	Se implementó anti-rebote por software verificando que hayan transcurrido al menos 250 ms desde el último cambio de estado antes de procesar un nuevo flanco de bajada.

3. Preguntas de Repaso

3.1. Si el voltaje del potenciómetro es 1 V, ¿qué valor de ADC representará?

Aplicando la ecuación (2):

$$\text{Valor ADC} = 1,0 \times \frac{4095}{3,3} = \frac{4095}{3,3} \approx \boxed{1241} \quad (6)$$

Este valor indica que a 1 V el ADC habrá utilizado 1241 de sus 4095 niveles disponibles, representando aproximadamente el 30,3 % del rango total de conversión.

3.2. ¿Cuál es la representación binaria de un voltaje de 2 V?

Primero se obtiene el valor decimal del ADC:

$$\text{Valor ADC} = 2,0 \times \frac{4095}{3,3} \approx \boxed{2482} \quad (7)$$

Convirtiendo 2482 a binario de 12 bits:

$$2482_{10} = 2048 + 256 + 128 + 32 + 16 + 2 = \boxed{100110110010_2} \quad (8)$$

Verificación: $2^{11} + 2^8 + 2^7 + 2^5 + 2^4 + 2^1 = 2048 + 256 + 128 + 32 + 16 + 2 = 2482 \checkmark$

3.3. ¿Por qué se utiliza 4095 y no 4096 en la ecuación del ADC?

Un ADC de 12 bits puede generar $2^{12} = 4096$ niveles distintos de cuantización. Sin embargo, el rango de estos niveles comienza en **0** (no en 1), de modo que los valores posibles son $\{0, 1, 2, \dots, 4095\}$. El valor máximo es por lo tanto $4096 - 1 = \mathbf{4095}$.

Al utilizar 4095 como denominador en la ecuación (1), se asegura que cuando el ADC lea su valor máximo (4095), el resultado sea exactamente 3,3 V:

$$V = 4095 \times \frac{3,3}{4095} = 3,3 \text{ V} \checkmark \quad (9)$$

Si se usara 4096 como denominador, el valor máximo daría $4095 \times (3,3/4096) \approx 3,2992 \text{ V}$, introduciendo un error sistemático en toda la escala. El razonamiento es análogo para cualquier ADC de n bits: siempre se usa $2^n - 1$ como denominador.

3.4. ¿Cuál es la resolución de un ADC de 16 bits?

Un ADC de 16 bits tiene $2^{16} = 65536$ niveles, por lo que su resolución con referencia de 3,3 V es:

$$\text{Resolución}_{16} = \frac{3,3 \text{ V}}{2^{16} - 1} = \frac{3,3}{65535} \approx \boxed{0,0504 \text{ mV por nivel}} \quad (10)$$

Comparado con el ADC de 12 bits de la Pico W ($\approx 0,806 \text{ mV/nivel}$), el ADC de 16 bits ofrece una resolución aproximadamente **16 veces mayor**, siendo adecuado para medir variaciones muy pequeñas de señal.

3.5. Conversión de valores binarios del ADC a voltaje

Para cada valor binario se aplica el proceso: binario $\rightarrow$ decimal $\rightarrow$ voltaje mediante la ecuación (1).

Cuadro 2: Conversión de valores binarios ADC a voltaje equivalente

Binario (12 bits)	Decimal	Cálculo	Voltaje (V)
110111010101	3541	$3541 \times \frac{3,3}{4095}$	2,8536 V
001001100101	613	$613 \times \frac{3,3}{4095}$	0,4940 V
000010101111	175	$175 \times \frac{3,3}{4095}$	0,1410 V
110101101101	3437	$3437 \times \frac{3,3}{4095}$	2,7697 V

Procedimiento detallado para 110111010101:

$$\begin{aligned}
 110111010101_2 &= 2^{11} + 2^{10} + 2^8 + 2^7 + 2^6 + 2^4 + 2^2 + 2^0 \\
 &= 2048 + 1024 + 256 + 128 + 64 + 16 + 4 + 1 = 3541_{10} \\
 V &= 3541 \times \frac{3,3}{4095} \approx 2,8536 \text{ V}
 \end{aligned} \tag{11}$$

3.6. Funcionamiento del Código C/C++ (Arduino)

El código se estructura alrededor de un **bucle principal no bloqueante** que mantiene tres tareas concurrentes usando `millis()`:

- **Lectura del ADC:** Cada `SAMPLE_MS` (400 ms) se invoca `analogRead(A0)`, que internamente activa el ADC0 de la Pico W, realiza la conversión de 12 bits y devuelve un entero entre 0 y 4095. Este valor se convierte a voltaje con la ecuación (1) y se almacena en un **buffer circular** de 60 posiciones para la gráfica web.
- **Control del LED:** El intervalo de parpadeo se recalcula en cada iteración a partir del último valor ADC leído. Si el tiempo transcurrido supera dicho intervalo, el estado del LED se invierte con `digitalWrite()`. Al activarse el modo HOLD, el LED permanece encendido fijo hasta que se vuelve a presionar el botón.
- **Servidor HTTP:** `server.handleClient()` procesa cualquier petición entrante. La ruta `/` devuelve el HTML de la interfaz en cuatro fragmentos (*chunked transfer*) para no saturar la RAM. La ruta `/data` devuelve un JSON pequeño con el voltaje actual, el valor ADC, el intervalo del LED y el arreglo de datos históricos, que el navegador consulta automáticamente cada 500 ms para actualizar la gráfica.

La detección del botón se realiza comprobando `digitalRead(BTN_PIN)` en cada iteración. Gracias al **anti-rebote por software** (250 ms de ventana muerta), un solo clic físico se traduce en exactamente un cambio de estado de la variable `paused`.

4. Conclusión

Esta práctica consolidó la comprensión del proceso de conversión analógica a digital, desde los fundamentos matemáticos de la cuantización y codificación hasta su implementación concreta en la Raspberry Pi Pico W. La resolución de $\approx 0,8\text{ mV}$ por nivel del ADC de 12 bits demostró ser suficiente para detectar variaciones suaves del potenciómetro, visibles tanto en el monitor serial como en la gráfica web.

La integración del servidor HTTP embebido añadió una dimensión práctica importante: la necesidad de optimizar el uso de memoria del microcontrolador (envío *chunked* del HTML) y de diseñar un bucle principal que atienda simultáneamente la red, el ADC y los periféricos sin recurrir a bloqueos con `delay()`. El botón HOLD demostró cómo una funcionalidad sencilla de hardware puede ofrecer una experiencia de usuario significativa cuando se combina con una interfaz visual.

Posibles mejoras:

- **Múltiples canales ADC:** Leer simultáneamente GP26, GP27 y GP28 para mostrar tres señales independientes en la misma gráfica web.
- **Almacenamiento de datos:** Implementar un endpoint `/csv` que permita descargar el historial completo de lecturas para análisis posterior.
- **Filtrado digital:** Aplicar un filtro de media móvil sobre las muestras del ADC para reducir el ruido eléctrico inherente al potenciómetro y al cable de señal.
- **Escala ajustable:** Agregar controles en la página web para modificar la frecuencia de muestreo o el rango visible de la gráfica sin necesidad de reprogramar la placa.

Anexos

Anexo A – Código Fuente Completo (main.cpp)

```

1  #include <Arduino.h>
2  #include <WiFi.h>
3  #include <WebServer.h>
4
5  #define WIFI_SSID "TU_SSID"
6  #define WIFI_PASS "TU_PASSWORD"
7
8  #define POT_PIN    A0      // GP26
9  #define LED_PIN    14      // GP14
10 #define BTN_PIN    15      // GP15
11
12 #define ADC_BITS    12
13 #define ADC_MAX     4095
14 #define ADC_VREF     3.3f
15 #define BLINK_MIN_MS 50

```

```

16 #define BLINK_MAX_MS 1000
17 #define BUF_SIZE     60
18 #define SAMPLE_MS    400
19
20 WebServer server(80);
21
22 float      voltBuf[BUF_SIZE];
23 int        bufIdx   = 0;
24 int        bufCount = 0;
25 bool       paused   = false;
26 bool       ledState = false;
27 uint16_t   rawADC    = 0;
28 float      voltage  = 0.0f;
29 uint32_t   lastBlinkMs = 0;
30 uint32_t   lastSampleMs = 0;
31 uint32_t   lastBtnMs  = 0;
32
33 uint32_t   adcToBlinkInterval(uint16_t adc) {
34     float norm = (float)adc / (float)ADC_MAX;
35     return (uint32_t)(BLINK_MAX_MS - norm * (BLINK_MAX_MS -
        BLINK_MIN_MS));
36 }
37
38 void handleRoot() {
39     server.setContentLength(CONTENT_LENGTH_UNKNOWN);
40     server.send(200, "text/html; charset=UTF-8", "");
41     server.sendContent("<!DOCTYPE html><html lang='es'><head>"
42         "<meta charset='UTF-8'><title>ADC Monitor</title>"
43         "<script src='https://cdn.jsdelivr.net/npm/chart.js@4.4.0/"
44             "dist/chart.umd.min.js'></script>"
45         "<!-- Estilos y tarjetas omitidos por brevedad -->"
46         "</head><body>");
47     server.sendContent("<!-- Grafica y logica JS omitidos por
        brevedad -->");
48     server.sendContent("</body></html>");
49     server.sendContent("");
50 }
51
52 void handleData() {
53     String json = "{ \"paused\": " + String(paused ? "true" : "
        false");
54     json += ", \"voltage\": " + String(voltage, 3);
55     json += ", \"adc\": " + String(rawADC);
56     json += ", \"interval\": " + String(adcToBlinkInterval(rawADC)
        );
57     json += ", \"data\": [";
58     for (int i = 0; i < bufCount; i++) {
59         int idx = (bufIdx - bufCount + i + BUF_SIZE) % BUF_SIZE;

```

```
59     json += String(voltBuf[idx], 3);
60     if (i < bufCount - 1) json += ",";
61 }
62 json += "]}";
63 server.setHeader("Access-Control-Allow-Origin", "*");
64 server.send(200, "application/json", json);
65 }
66
67 void setup() {
68     Serial.begin(115200);
69     delay(2000);
70     analogReadResolution(ADC_BITS);
71     pinMode(LED_PIN, OUTPUT);
72     digitalWrite(LED_PIN, LOW);
73     pinMode(BTN_PIN, INPUT_PULLUP);
74     WiFi.begin(WIFI_SSID, WIFI_PASS);
75     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.
        print("."); }
76     Serial.print("\nIP: http://"); Serial.println(WiFi.localIP()
        );
77     server.on("/", handleRoot);
78     server.on("/data", handleData);
79     server.begin();
80 }
81
82 void loop() {
83     server.handleClient();
84     uint32_t now = millis();
85     // Boton HOLD
86     if (digitalRead(BTN_PIN) == LOW && (now - lastBtnMs > 250))
87     {
88         paused = !paused;
89         lastBtnMs = now;
90         if (paused) digitalWrite(LED_PIN, HIGH);
91     }
92     if (!paused) {
93         // Muestreo ADC
94         if (now - lastSampleMs >= SAMPLE_MS) {
95             rawADC = (uint16_t)analogRead(POT_PIN);
96             voltage = (rawADC * ADC_VREF) / (float)ADC_MAX;
97             voltBuf[bufIdx] = voltage;
98             bufIdx = (bufIdx + 1) % BUF_SIZE;
99             if (bufCount < BUF_SIZE) bufCount++;
100             lastSampleMs = now;
101         }
102         // Parpadeo LED
103         uint32_t interval = adcToBlinkInterval(rawADC);
104         if (now - lastBlinkMs >= interval) {
```

```
104     ledState = !ledState;
105     digitalWrite(LED_PIN, ledState ? HIGH : LOW);
106     lastBlinkMs = now;
107   }
108 }
109 }
```

Anexo B – Archivo platformio.ini

```
1 [env:rpipicow]
2 platform = https://github.com/maxgerhardt/platform-raspberrypi
3           .git
4 board = rpicow
5 framework = arduino
6 board_build.core = earlephilhower
7 monitor_speed = 115200
```

Anexo C – Salida del Monitor Serial

```
1 [WiFi] Conectando a 'MiRed' .....
2 [WiFi] Conectado!
3 [WiFi] Abre en tu navegador: http://192.168.1.105
4 [HTTP] Servidor iniciado en puerto 80.
5 ADC: 2048 | Voltaje: 1.650 V | Intervalo LED: 525 ms
6 ADC: 3100 | Voltaje: 2.497 V | Intervalo LED: 244 ms
7 ADC: 4050 | Voltaje: 3.264 V | Intervalo LED: 54 ms
8 [HOLD] ADC: 4050 V: 3.264
9 ADC: 150 | Voltaje: 0.121 V | Intervalo LED: 982 ms
10 [ACTIVO] Reanudando...
```

Anexo D – Fotografía del Prototipo Armado

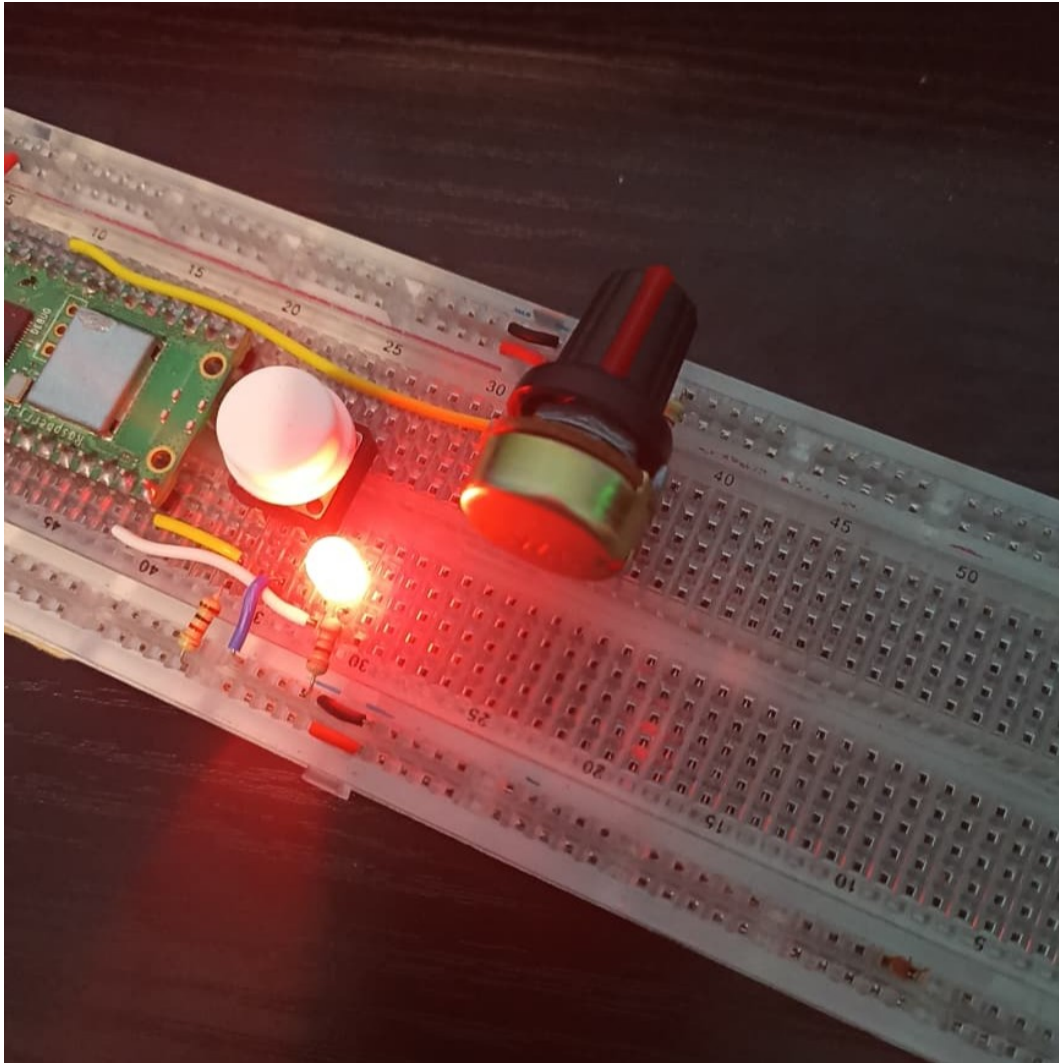


Figura 2: Fotografía del circuito armado en protoboard: Raspberry Pi Pico W con potenciómetro de $5\text{ k}\Omega$ conectado al canal ADC0 (GP26), LED en GP14 con resistencia limitadora de $220\ \Omega$ y pulsador en GP15 para el modo HOLD. La alimentación se realiza por USB para mantener activa la comunicación serial y el servidor WiFi.

Referencias

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.0). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [3] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.

- [4] PlatformIO Labs, *PlatformIO Documentation*. <https://docs.platformio.org>, 2024.
- [5] Chart.js Contributors, *Chart.js Documentation v4*. <https://www.chartjs.org/docs/latest/>, 2024.
- [6] lwIP Documentation, *A Lightweight TCP/IP Stack*. <https://savannah.nongnu.org/projects/lwip/>, 2024.